You then wrap the function `f` in a `std::packaged_task<result_type()>` ❸, because `f` is a function or callable object that takes no parameters and returns an instance of type `result_type`, as we just deduced. You can now get your future from the `std::packaged_task<>` ❹, before pushing the task onto the queue ❺ and returning the future ❻. Note that you have to use `std::move()` when pushing the task onto the queue, because `std::packaged_task<>` isn't copyable. The queue now stores `function_wrapper` objects rather than `std::function<void()>` objects in order to handle this.

This pool thus allows you to wait for your tasks and have them return results. The next listing shows what the `parallel_accumulate` function looks like with such a thread pool.

Listing 9.3. `parallel_accumulate` using a thread pool with waitable tasks